Virtualized Communication Controllers in Safety-Related Automotive Embedded Systems

Dominik Reinhardt(✉), Maximilian Güntner, and Simon Obermeir

BMW AG, Munich, Germany
{dominik.reinhardt,maximilian.guentner,simon.obermeir}@bmw.de

Abstract. Meeting non-functional requirements, like safety according to ISO26262, is gaining rising importance in the automotive industry along with the reuse and migration of existing applications. Embedded virtualization is a promising technology to isolate software and its possible faults. In a virtualized system shared communication devices constitute a bottleneck and require special treatment for safety-related systems.

In this paper, we evaluate the emulation of CAN and Ethernet hardware drivers in virtualized automotive software systems of BMW. We show a methodology how to relocate and isolate safety-related drivers within a large scale software integrated virtualized system. For our analysis we use the Infineon AURIX TriCore TC27x controller. To encapsulate our software, we use a research-based hypervisor supported by ETAS Ltd.

Keywords: Automotive · Embedded multicore · Embedded hypervisor · Virtualization · Emulated communication controller · CAN · Ethernet

1 Introduction

In the next decade, many new features and technologies, like Advanced Driver Assistant Systems (ADAS), will be integrated in our cars [3], [4]. The trend to develop bigger and more complex software systems on capable Electronic Control Units (ECUs) is rising. To tackle such always increasing amount of vehicle functionality, Electric and Electronic (E/E) architectures, using the example of BMW [12], were reordered in a domain-oriented manner. To reduce the amount of ECUs, automotive software is consolidated onto comprehensive hardware platforms. This methodology is illustrated in figure 1 and is called Large Scale Software Integration (LSSI) [12].

Virtualization technology could be a proper methodology to integrate software components in a transparent and flexible way, protected from each other [12], [14]. To be prepared for safety-related systems, the ISO26262 demands freedom from interference between vehicle software and functionality [8]. The isolation features between Virtual Machines (VMs) could achieve that requirement

© Springer International Publishing Switzerland 2015
L.M. Pinho et al. (Eds): ARCS 2015, LNCS 9017, pp. 173–185, 2015.
DOI: 10.1007/978-3-319-16086-3_14

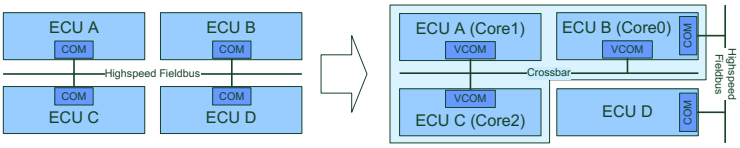


Fig. 1. Repartitioning of ECUs in automotive E/E systems

for separation of safety-related software parts from the rest of the system. The Hypervisor (HV) acts as the smallest instance related to all VMs in the system. For safety-related systems, the HV must be qualified to the highest Automotive Safety Integrity Level (ASIL) assigned to any application within the system. In any case, real-time capabilities, deterministic behavior and hard deadlines must be fulfilled to achieve required properties of an automotive embedded system.

In this paper, we focus on the emulation of communication controllers for Controller Area Network (CAN) and Ethernet in a paravirtualized automotive software system by using an embedded HV, closely developed to the AUTOSAR standard. We discuss how to permit peripheral access to VMs and how to deal with limited memory protection ranges. To run mixed-integrity automotive functionality, we suggest an approach to integrate the Communication Hardware Abstraction within the AUTOSAR Microcontroller Abstraction Layer (MCAL) in a decentralized manner, encapsulated in a unique VM. This method is analyzed for timing overheads of the information flow between VMs and interconnected automotive fieldbuses. For our analysis, we use an embedded HV with real-time capabilities. The hypervisor (called *RTA-HV*) is supported by ETAS Ltd. [13] and is ported to the Infineon AURIX TriCore microcontroller which fulfills our needs for paravirtualization [2].

The next sections are organized as follows: Section 2 outlines the related work on hardware emulation by using embedded HVs for real-time systems. Section 3 outlines methods to integrate systems in a paravirtualized environment. In section 4 we summarize the implementation to build virtual gateways for information routing. The results and performance evaluations are discussed in section 5. Section 6 summarizes the paper and its results.

2 Related Work

To achieve performance advantages, paravirtualization is already used in data centers supported by Virtual Machine Monitors (VMMs) like Xen [1]. Embedded real-time VMMs like XtratuM [9] or Sysgo's PikeOS are available and able to encapsulate safety-related functionality. But these HVs are not available for controllers without a Memory Management Unit (MMU) [2], which is mostly not integrated in state-of-the-art automotive devices. These are mostly equipped with only a Memory Protection Unit (MPU) to realize spatial protection.

Chip vendors like Intel or AMD feature their controllers with virtualization extensions like Intel VT or AMD-V. These chips are applied in data centers, where software applications are massively consolidated on from server systems

decoupled from the hardware. For I/O virtualization, extensions like Intel VT-c (Virtualization Technology for Connectivity) come into consideration [7].

Research based solutions for self-virtualized CAN and Ethernet controllers for embedded real-time systems is shown in [6] and [11]. No hardware emulation is necessary and message arbitration is processed controller internally. An approach without a full HV for CAN virtualization with safety aspects is presented in [14].

The architecture of our work is derived from Xen [1]. Front- and backend drivers called *Netfront* and *Netback* were used to pass information between the master VM called *dom0* and other guest VMs called *User Domains (DomUs)* (see figure 3). The *dom0* runs all device drivers needed from other *DomUs*.

3 Methods to Integrate Virtualized Peripherals Using the Infineon AURIX TriCore

The HV represents a Type-1 VMM which runs bare-metal in supervisor mode on the Infineon AURIX microcontroller. No hosting Operating System (OS) is needed. Every VM operates in a one-to-one mapping per core in user mode and implements its core local HV instance. Additional Virtual Device Emulators (VDEs) run in the context of the HV and have kernel access rights. VDEs allow the emulation of peripherals or the implementation of communication channels between VMs. Each VDE implementation exists once for the overall system and is re-entrant for any core and VM. A trap class 6 (System Call) is used to trigger the HV. Every VDE implements its own *request()* function, where incoming service requests are processed. VDEs possess the highest interrupt priority levels in the system. If the core is handling a trap routine or executing HV code, all interrupts for VMs are blocked.

3.1 Granting Access to Peripheral Space

For the Infineon TriCore AURIX microcontroller, three different privileged modes exist. In supervisor mode there are no restrictions for memory or peripheral access. In user mode, two distinctions exist between User-0 mode, where peripheral access is generally restricted and User-1 mode which basically grants access to I/O controllers. Any register call of a VM which runs in User-0 or 1 mode is either allowed due to the Instruction Set Architecture (ISA) or must be executed by the HV which runs in Supervisor (SU) mode itself. For the latter, the HV verifies such calls on software level. If they are not allowed, they will be restricted by the MPU. To grant access to communication controllers or to other peripherals, we identified the following three use cases. To operate virtual communication controllers, all three approaches are applicable and must be analyzed depending on the project situation and hardware resources.

1. **Direct Device Assignment (DDA)**: DDA is a well proven technique like for server systems in data centers. Peripherals like CAN or Ethernet controller must not be shared between multiple virtual machines. The VM has

to run in User-1 mode and affected peripheral address space must be registered additionally in the MPU per core. For sure, this method is restricted by given MPU protection ranges and by given hardware resources.

2. **Paravirtualization of non-shared devices**: If the MPU's protection address ranges are insufficient to grant access to peripheral space, hardware accesses must be paravirtualized. In this case, every call from VMs must be wrapped by HV and forwarded to the HV and its VDEs. This is mandatory if the virtual machine runs in User-0 mode. Unfortunately, this approach will increase the timing overhead due to trap and emulation due to the HV.
3. **Emulation of shared devices**: In this case, more than one virtual machine needs access to a particular peripheral. The access must be controlled and arbitrated by the HV. There are identical timing overheads as in use case 2 and besides additional efforts for routing, concurrent accesses and blocking times must be taken into account. To arbitrate messages within the HV an additional VDE must be developed.

3.2 Relocating Hardware Drivers to Allow Safe and Secure Accesses

Every peripheral needs its individual hardware drivers for operation located in the AUTOSAR MCAL. Typically, software instances should have exclusive access to peripherals. Within LSSI systems probably some peripherals must be shared. For mixed-integrity systems it is problematic to introduce unqualified software and its communication channels into a safety-related system. The uncontrolled sharing of peripherals between software partitions without any controlling instance can lead to malfunctions [8], [14]. Especially, in case of VM shutdown or restart, hardware drivers for other VMs must always be available for safety-related software parts. Therefore, we list identified approaches for paravirtualized automotive E/E systems, illustrated in figure 2.

1. **Driver modules integrated within each VM**: All drivers are integrated within a VM and run completely in user space. Either the VM has direct access to peripherals, due to the AURIX' User-1 mode granted by the MPU, or all attempts to register access have to be executed within kernel space by the HV. In any case, the VM needs exclusive access to peripherals (DDA). All SU only registers must be handled from the HV by using hypercalls. We use a

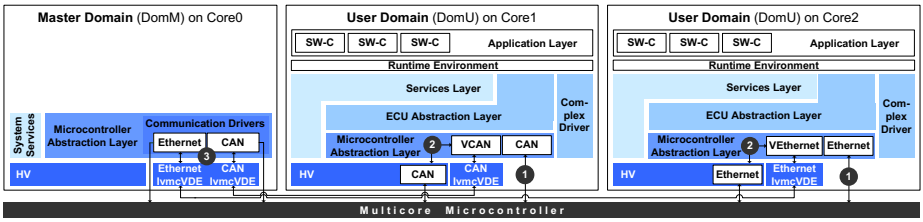


Fig. 2. Possible relocation for hardware drivers in an AUTOSAR system

paravirtualized approach where trapping and emulation of register calls are reduced as much as possible and routed to the HV.

2. **Driver modules integrated within the HV**: Relevant MCAL modules are located within the HV and operate in kernel space in SU mode. They have direct access to the hardware and all privileges. In this configuration, malicious drivers can jeopardize the overall system (single point of failure), even the HV itself. All attempts to have access to peripherals must be routed to and arbitrated by a VDE. This will increase the timing overheads because of trap and emulation to reach the HV.
3. **Driver modules relocated to a privileged VM**: This approach differs between guest VMs called DomUs and a unique VM called Master Domain (DomM). This method is very similar to Xen's *dom0* approach [1], where all drivers run in a special domain. DomM integrates drivers in a special VM. Only DomM routes frames to the fieldbus and has exclusive access to the communication controller. All attempts to exchange data of DomUs must be transmitted to DomM and routed to another DomU or fieldbus. The DomM must not run in supervisor mode. To gain performance advantages, exclusive access to peripherals (DDA) which run in User-1 mode is necessary. The DomM runs as an encapsulated VM and implements a bridge for incoming attempts to peripheral access. It routes I/O information to VMs or fieldbuses.

4 Building Virtual Gateways

Sharing peripherals during run-time will cause interference to arbitrate frames (time) and protect peripheral access (space). We focus on the supervised emulation of communication controllers in safety-related systems. DDA needs no special routing mechanism because all devices are assigned exclusively to VMs. But this method is only possible if no peripheral must be shared. If drivers are relocated to the HV, its Trusted Computing Base (TCB) will increase. Due to the timing limitations and arbitration overheads within HV withdraws that method, as well. Therefore, we analyze the relocation of drivers to an independent, priviledged VM called DomM, as described in section 3.2. To grant peripheral access for hardware drivers (see section 3.1), MPU regions can be used exclusively for that purpose. Because of exclusive access to peripherals in DomM there is no need for hardware emulation (DDA).

4.1 Information Exchange Between Virtual Machines

To guarantee a safe and secure way to interact between VMs and cores, we implement a special VDE interface called Inter-Virtual-Machine-Communication VDE (IvmcVDE) which acts as a generic communication channel to transfer data [10]. Every information flow between VMs is under the control of the HV itself. The HV forwards incoming service requests to its IvmcVDE. Either polling or interrupt-driven communication mechanisms are possible to exchange information between VMs. Both strategies implement an event channel between VMs. All data is exchanged by using HV mechanisms like IvmcVDE.

In case of polling strategies, the VM has to notify new information by observing global data memory sections in the Local Memory Unit (LMU) of the AURIX. No traps or interrupts are needed to activate other cores. If there is new information available in the LMU, the dedicated VM will fetch that information periodically. In case of interrupt-driven strategies, we use a notification mechanism supplied by the HV, to trigger other cores and VM instances to re-enter the context of IvmcVDE. The sender core stores data in the global data RAM and triggers the receiver core by using an interrupt. The *request()*-function of IvmcVDE is reached to process new data.

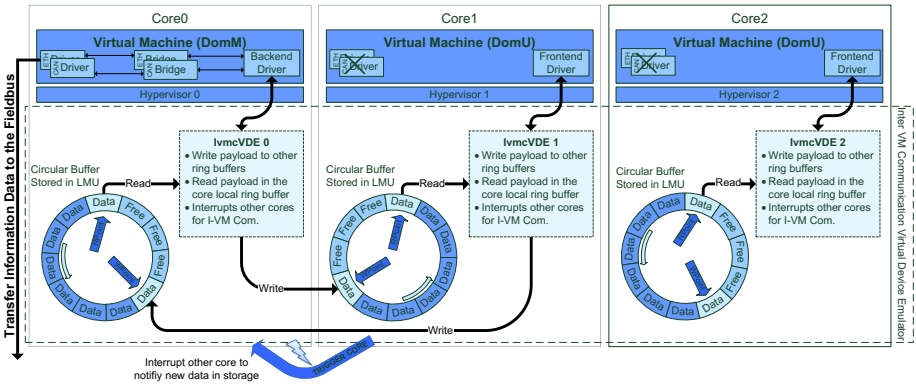


Fig. 3. Inter virtual machine communication mechanism build in a VDE

To avoid blocking accesses or starvation we implement three ring buffers (see figure 3) by using the *Circular Buffer Addressing Mode* of the AURIX TriCore. This feature supports atomic load and store operations including circular pointer shifting. They are supplied hardware features of the AURIX microcontroller. For each buffer we choose a size of 4096 byte located in the global data RAM of the LMU. All DomUs write messages to the DomMs ring buffer where they are retrieved and routed (FIFO). Two types of event channels are implemented: The polling (a flag signals new data) and interrupt-driven mechanisms.

Figure 4 describes the base frame-formats in our system. The Protocol Control Information (PCI) of every frame is stored in an additional header. The *Request Mode* controls the IvmcVDE operation and distinguishes between transmission modes like polling or interrupt-driven strategies. The *Destination* and *Source Core* field directs the information exchange between VMs. Because we support dynamic package length to efficiently support variable frame length in IEEE 802.3 Ethernet, the *Length* field stores the overall payload size. The *Payload* includes a wrapped communication frame e.g. CAN or Ethernet. Since we are in a safe environment where data corruption is very unlikely due to ECC-RAM and lock-step cores, we can omit check-summing the payload. They are

Table 1. Exemplary routing table to direct CAN frames

ID	**Port**($DomM_{core0}$)	**Port**($DomU_{core1}$)	**Port**($DomU_{core2}$)
0x1F	TRUE	FALSE	TRUE
0xA2	FALSE	TRUE	TRUE
...	...	...	...

calculated and inserted within the hardware driver or the controller itself and must not be transmitted in that layer.

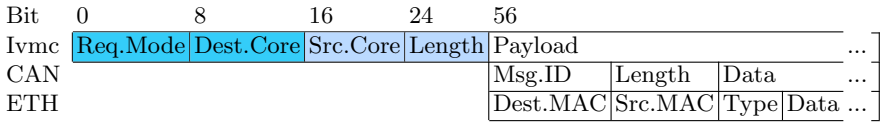


Fig. 4. Base Frame-Format for CAN and Ethernet

4.2 Routing Information Using a Privileged Domain

The DomM needs a scheduling scheme to route information between DomUs and other fieldbuses. Therefore, we integrate a bridging mechanism into DomM to route information ECU internally and externally. DomM implements a bridging functionality (see figure 3) including a Source Address Table (SAT) for Ethernet and a routing table for CAN which hold the routing protocols for incoming messages. A bridge typically acts on layer two of the ISO/OSI-model and routes information. Our VM bridge is divided into a controlling and a forwarding part:

The *controlling part* holds the routing table (see table 1) and the organization of it. In case of routing Ethernet frames there is a SAT (identical to a network switch) containing the MAC addresses which are assigned to connecting ports [5]. In case of CAN communication the routing table contains the CAN IDs. Therefore, we differentiate between a statically (CAN) and a dynamically (Ethernet) construction of our routing table (the SAT). The CAN routing table is configured statically before compile time and holds all routing information in advance. The Ethernet SAT will be setup dynamically during system run-time and assigns MAC addresses to (port) *IDs*.

The *forwarding part* routes messages between VMs or connecting fieldbuses (e.g. CAN or Ethernet). All incoming messages will be accepted, either by polling strategies or retrieved by an Interrupt Service Routine (ISR). We serialize messages and use the given IvmcVDE to exchange information between VMs. To transmit frames to a connected fieldbus, DomM implements needed hardware drivers, which have exclusive peripheral access to needed hardware controllers.

Table 1 lists an exemplary routing protocol for incoming CAN frames. The bridge distinguishes according to the *ID* (the CAN identifier) to which *Port* (the

VM and its core) the message must be routed through (e.g. message ID 0xA2 is routed to $core_1$ and $core_2$). If an incoming frame is assigned for DomM, it will be transferred to the connected fieldbus. All routing information is configured before compile time. In case of incoming Ethernet frames, the bridge distinguishes according to the *ID* (the MAC address) to which *Port* (the VM and its core) the message must be routed through. Similar to the CAN approach, it assigns multiple MAC addresses (*IDs*) to *Ports*.

5 Results and Discussion

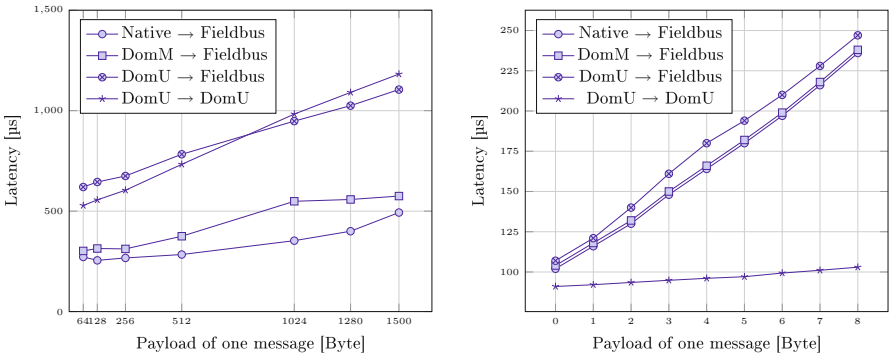
We want to evaluate the overhead, which occurs due to paravirtualization of the system and relocating the hardware drivers, with the different approaches mentioned above. Therefore, we compare a paravirtualized system to a native system version. Our measurements are executed on the Infineon AURIX TriCore TC27X [2]. Each calculation core operates at a clock rate of 200 MHz. The focus is on peripheral usage like HighSpeed-CAN (500 kbit/s) and Fast Ethernet (100 Mbit/s) communication controller. For all our measurements, each core is exclusively used for sending or receiving messages. Blocking times due to concurrent accesses are avoided. To be close to real platforms, all measurements are realized multiple times with external clients and include the transmission overheads of peripherals to a connected fieldbus.

Figure 5 and 6 are organized as follows: *Native* systems are not paravirtualized and represent the benchmark for our measurements. For *DomM→Fieldbus*, frames were transmitted directly from the privileged domain to the connecting fieldbus including all routing efforts. In *DomU→Fieldbus* scenario, frames were transmitted from a DomU to DomM and from this point routed to the fieldbus. In *DomU→DomU* scenario there is no limiting fieldbus involved and frames were exchanged ECU internally from a DomU routed by DomM to another DomU.

5.1 Response Time per Message Size

We want to analyze the response times of single messages for relocated automotive communication drivers. Figure 5 shows the timing measurements to transmit messages over a fieldbus compared by the size of their payload by using interrupt-driven communication mechanisms. After message routing by DomM we use DDA to access peripherals directly and have exclusive access to relevant controller registers. With it, we avoid processing overhead due to the HV for data transmission. For interrupt-driven data reception, there is as static timing overhead of 4 µs to route incoming interrupts to VMs [13]. Thus, for all our measurements the timing gaps between the native and DDA case is minimalistic.

Figure 5a shows the transmission of Ethernet frames using the ICMP protocol to measure echo replies of different sizes (*ping*). The message exchange of bigger frames has more impact on latency compared to CAN. This is reasonable, because copy operations of bigger frame structures are more calculation



(a) Transmission of Ethernet frames. (b) Transmission of CAN frames.

Fig. 5. Response time measurements compared by their payload

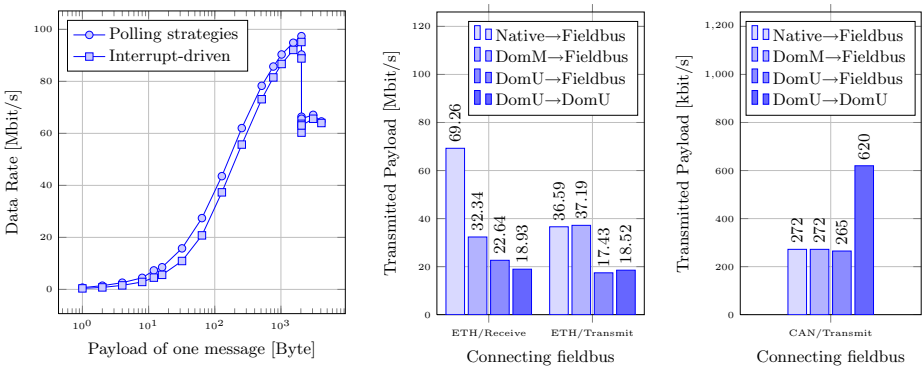
intensive and cause additional timing overheads. Routing frames from one VM to another need two copy operations, which need more time if the payload size increases. In case of transmitting frames from DomU to DomM, there is firstly additional management overhead for information exchange between VMs using IvmcVDE. Secondly, the calculation effort due to OSI abstraction layers for TCP and message routing (using a SAT) by each VM (compare the gap between *DomM→Fieldbus* and *Native→Fieldbus*). Sending Ethernet frames of 64 byte from any DomU to the fieldbus takes 620 µs. The transmission of 1500 byte of payload takes not even the double amount of transmission time. At around 700 byte of payload, we measured the break-even point of latency to exchange Ethernet frames between DomUs or transmit them over the fieldbus.

The CAN response time analysis is illustrated in figure 5b and shows a similar direct proportional trend for all measurements. Because of small CAN frame sizes, copy operations between VMs are not mentionable which relate to a slightly increasing trend. The management overhead for routing frames in DomM, using a lookup table with a time complexity of $\mathcal{O}(1)$, causes timing overhead around 1 µs. The transmission of frames from any DomU by using IvmcVDE causes additional timing overhead, around 4%. The longest time is required by the CAN controller itself to transmit frames over the fieldbus. The time to instantiate a frame until it is handed over to the controller is less than 5 µs. The remaining time is used for the frame transmission within the CAN controller. Because of small CAN frame sizes the transmission overheads between VMs caused by IvmcVDE are negligible and even faster than the connecting CAN fieldbus. We determine that information routing in an optimal paravirtualized system has nearly no impact on response times and achieves nearly linear measurement results.

5.2 Maximum Throughput Measurements

First, we will analyze the possible bandwidth to transfer data between VMs. Therefore, we will measure the data rates from DomU to DomM, which is pre-

sented in figure 6a. No routing overheads incur in this scenario. We can achieve slightly higher data rates with polling than with interrupt-driven mechanisms. In case of using interrupt-driven mechanisms there are more processing efforts necessary. This is because all interrupts must be routed through the HV due to less virtualization support of the hardware. Basically, the data rate is lower for the transmission of smaller frame sizes than for bigger frame sizes. This relates to fixed processing overheads to handle frames within the HV and between VMs. Copy operations (load and store) of bigger data types in sequence are more efficient than smaller data frames (e.g. 7 byte of payload) which must be copied byte per byte. Our ring buffers have a size of 4096 byte. The data rate drops at 2049 byte (including 6 byte of PCI). For this message size the ring buffer cannot store more than one frame at once. Then only sequential read and write operations are possible. Our tests show that the IvmcVDE is capable to transmit data up to 97 Mbit/s, with 2047 byte message size, from one VM to another.



(a) Max rate between VMs. (b) Max. rate for ETH. (c) Max. rate for CAN.

Fig. 6. Measured data rates between VMs by using the IvmcVDE

Second, we analyze the maximum data-throughput (interrupt-driven) for CAN and Ethernet. Ethernet measurements (*full-duplex*) are illustrated in figure 6b. Native systems without virtualization (*Native→Fieldbus*) have a doubled reception rate compared by the rate to transmit data. This is reasonable, because no data copies (*zero-copy*) are necessary to receive frames. In addition, incoming frames are allocated per Direct Memory Access (DMA) in the VMs data structures. To transmit frames, each message must be copied once into the data structure of the Ethernet controller individually. In case of *DomM→Fieldbus* the reception rate is nearly cut in half because of an additional copy operation in the bridge of DomM. In case of *DomU→Fieldbus* the data is received by DomM and first copied from IvmcVDE into the ring buffer located in the context of the receiving VM. Here, the data is transmitted to the dedicated DomU which will cause the second data copy. To transmit data to a fieldbus, even a third copy operation into the data structure of the Ethernet controller is necessary. Therefore, to achieve high data-rates with Ethernet, the amount of copy operations

by using a DomM has a significant influence on communication performance of smaller embedded controllers. Compared, to native systems we can transfer less data between VMs, either in case of *DomU→Fieldbus* or *DomU→DomU*, due to additional time-consuming copy operations of payload sizes mostly around 1500 byte. Summarized, there are timing penalties to bridge Ethernet frames in a paravirtualized system and using a DomM by using limited hardware.

For CAN, the maximum transmission rates are limited by the speed of the transmitting hardware controller and by the fieldbus bitrate illustrated in figure 6c. There are no maximum reception rates for because we are limited to the bitrate of the CAN fieldbus with only one active transmitter at once. We can receive CAN frames much faster than transmitting them and determined no frame losses. In the *native* case without virtualization, we achieve a maximum transmission data rate of 272 kbit/s with a fixed payload of 8 byte per frame. Including its PCI, a typical CAN frame has a size of 108 bit which now achieves an overall data rate of 459 kbit/s. We obtain the same ratio for *DomM→Fieldbus* including all routing efforts. This relates to the much slower transmission time of the CAN controller itself for coping data to any data structures in the MCAL (see section 5.1). In case of *DomU→Fieldbus* we achieve similar results. The slight deviation is due to management overhead in the IvmcVDE for message exchange between VMs. To transmit full CAN frames between DomUs by using a managing DomM (*DomU→DomU*) we measure reception timings around 103 µs which is much faster than sending them over the CAN fieldbus. Therefore, we can obtain higher data rates with a maximum throughput of 620 kbit/s. Summarized, even by usage of a decentralized DomM, software-driven CAN routing within a paravirtualized system has no big impact on the overall performance.

6 Conclusion

In this paper, we showed an approach to relocate hardware drivers of communication controllers like CAN and Ethernet. Our goal was to run them in a safe execution space to decouple safety-related functionality from untrusted software parts. Additionally, the TCB of the HV should be kept as small as possible. Therefore, a privileged VM like DomM is a solution to encapsulate safety-related hardware drivers, isolated from non-qualified software, in coherence of an embedded HV. To achieve real-time and safety requirements, some tradeoffs must be taken into account. If additional communication paths and small timing efforts can be tolerated, we showed a methodology for building virtual gateways to route information between virtualized automotive embedded systems.

Ethernet for automotive environments is mostly used to transfer big data volumes. Due to additional copy operations the transmission rate is reduced between VMs compared to native systems. Achieving higher data rates would require faster hardware controllers with supporting virtualization extensions. CAN is used to transmit small chunks of data within a reasonable timeframe. According to our measurements, these requirements are realizable in our paravirtualized system. Furthermore, the achieved bandwidth between VMs is significant higher compared to any other automotive fieldbuses. If messages are

exchanged exclusively by ECUs, the migration to VMs can save bandwidth of connecting fieldbuses. Thus, the system migration to VMs within LSSI platforms could be an intermediate step for communication intensive ECUs. For safety-related systems, our approach, to encapsulate drivers in a safe environment, realizes an already proven method in data centers to separate hardware drivers from VMs.

Acknowledgments. The authors would like to express their gratitude to Michael Scheffer from Infineon Technologies AG for his cooperative work and for supplying the hardware drivers and frameworks. This work was funded within the project ARAMiS by the German Federal Ministry for Education and Research with the funding IDs 01IS11035. The responsibility for the content remains with the authors.

References

1. Barham, P., Dragovic, B., Fraser, K., Hand, S., Harris, T., Ho, A., Neugebauer, R., Pratt, I., Warfield, A.: Xen and the Art of Virtualization. SIGOPS Oper. Syst. Rev. **37**(5), 164–177 (2003)
2. Brewerton, S., Schneider, R.: Hardware based paravirtualization: simplifying the co-hosting of legacy code for mixed criticality applications. In: SAE 2013 World Congress & Exhibition, April 2013
3. Broy, M.: Challenges in automotive software engineering. In: Proc. ACM 28th International Conference on Software Engineering (ICSE 2006), pp. 33–42, Shanghai, May 2006
4. Fürst, S.: Challenges in the design of automotive software. In: Design, Automation Test in Europe Conference Exhibition (DATE), pp. 256–258, March 2010
5. Güntner, M.: Virtualization of Time Critical Communication Interfaces. Bachelor's thesis, University of Applied Sciences Kempten, September 2014
6. Herber, C., Richter, A., Rauchfuss, H., Herkersdorf, A.: Self-virtualized CAN controller for multi-core processors in real-time applications. In: Kubátová, H., Hochberger, C., Daněk, M., Sick, B. (eds.) ARCS 2013. LNCS, vol. 7767, pp. 244–255. Springer, Heidelberg (2013)
7. Intel LAN Access Division. Intel VMDq Technology. White paper, Intel Corporation, March 2008. Revision 1.2
8. ISO 26262. Road vehicles - Functional safety - Part 1–10 (2011)
9. Masmano, M., Ripoll, I., Crespo, A., Metge, J.J.: XtratuM: a hypervisor for safety critical embedded systems. In: Eleventh Real-Time Linux Workshop, Dresden, Germany, September 2009
10. Obermeir, S.: Integration of Virtualized AUTOSAR-Systems on a Multicore-Controller. Master's thesis, University of Applied Sciences Munich, October 2014

11. Rauchfuss, H., Wild, T., Herkersdorf, A.: A network interface card architecture for i/o virtualization in embedded systems. In: Proceedings of the 2Nd Conference on I/O Virtualization, WIOV 2010, p. 2. USENIX Association, Berkeley (2010)
12. Reinhardt, D., Kaule, D., Kucera, M.: Achieving a Scalable E/E-Architecture using AUTOSAR and Virtualization. SAE International Journal of Passenger Cars - Electronic and Electrical Systems **6**(2), 489–497 (2013)
13. Reinhardt, D., Morgan, G.: An embedded hypervisor for safety-relevant automotive E/E-systems. In: 2014 9th IEEE International Symposium on Industrial Embedded Systems (SIES), pp. 189–198, June 2014
14. Schneider, R., Kohn, A., Schmidt, K., Schoenberg, S., Dannebaum, U., Harnisch, J., Zhou, Q.: Efficient virtualization for functional integration on modern microcontrollers in safety-relevant domains. In: SAE 2014 World Congress & Exhibition, April 2014